第9章
閾值處理邁向數位情報

9-1：threshold( )函數

- Threshold稱為臨界值，也稱閾值或門檻值。可用與像素值進行比較做差異化處理
- ret, dst = threshold(src, thresh, maxval, type)

❑ src：原始的影像，可以是灰階或彩色影像。

❑ thresh：閾值。

❑ maxval：需要設定像素的最大值。

❑ type：代表閾值函數處理的方法，可以參考下表。

具名常數	值	說明
THRESH_BINARY	0	影像值大於閾值取最大值，其他是 0
THRESH_BINARY_INV	1	影像值大於閾值取 0，其他取最大值
THRESH_TRUNC	2	影像值大於閾值取閾值，其他值不變
THRESH_TOZERO	3	影像值大於閾值則不變，其他取 0
THRESH_TOZERO_INV	4	影像值大於閾值取 0，其他值不變
THRESH_OTSU	8	使用演算法自動計算閾值
THRESH_TRIANGLE	16	使用三角形演算法自動計算閾值

二值化處理THRESH_BINARY與現代情報戰

- 二值化是圖像分割的一種最簡單的方法。二值化可以把灰階影像轉換成二值圖像。把大於某個臨界灰階值的像素值設為某個極大值，把小於這個值的像素灰度設為極小值，將影像轉換成二值化像素值。

- 根據閾值選取的不同，二值化的算法分為固定閾值和自適應閾值
- if 像素值 > 閾值:
- 　　像素值 = 最大值
- else:
- 　　像素值 = 0

- 程式實例：使用隨機函數建立3 x 5陣列，然後將大於127的值設為255，其他值設為0，讀者可以由此了解threshold( )函數的用法。
  - import cv2
  - import numpy as np

  - thresh = 127 # 定義閾值
  - maxval = 255 # 定義像素最大值
  - src = np.random.randint(0,256,size=[3,5],dtype=np.uint8)
  - ret, dst = cv2.threshold(src,thresh,maxval,1)
  - print(f"src =\n {src}")
  - print(f"threshold = {ret}")
  - print(f"dst =\n {dst}")

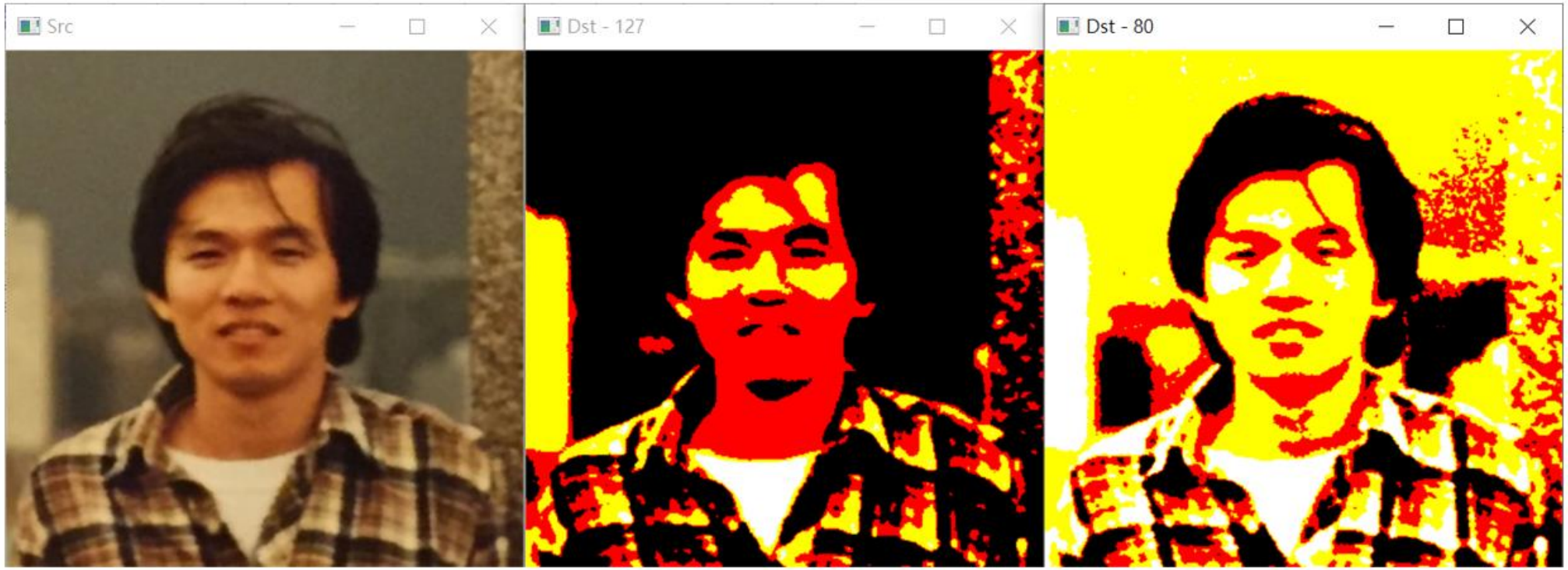
```
src =
 [[ 70 216 239  24  70]
 [230  22 216 244 214]
 [ 37 236  51  68  54]]
threshold = 127.0
dst =
 [[  0 255 255   0   0]
 [255   0 255 255 255]
 [  0 255   0   0   0]]
```

- 程式實例ch9_2.py：將影像用灰階讀取，然後分別設定閾值為127和80，了解可能的結果。(0 black ； 255 white)

threshold = 127　　threshold = 80

```
import cv2

thresh = 127                    # 定義閾值
maxval = 255                    # 定義像素最大值
src = cv2.imread("jk.jpg",cv2.IMREAD_GRAYSCALE)
ret, dst = cv2.threshold(src,thresh,maxval,cv2.THRESH_BINARY)  #ret門檻值; dst處理後影像
print (src)
print (dst)
cv2.imshow("Src",src)
cv2.imshow("Dst - 127",dst)        # threshold = 127
thresh = 80                     # 修訂所定義的閾值
ret, dst = cv2.threshold(src,thresh,maxval,cv2.THRESH_BINARY)
cv2.imshow("Dst - 80",dst)         # threshold = 80

cv2.waitKey(0)
cv2.destroyAllWindows()
```

- 程式實例：使用彩色讀取，分別以threadhold設為127與80，並與原圖比較其結果



threshold = 127　　threshold = 80

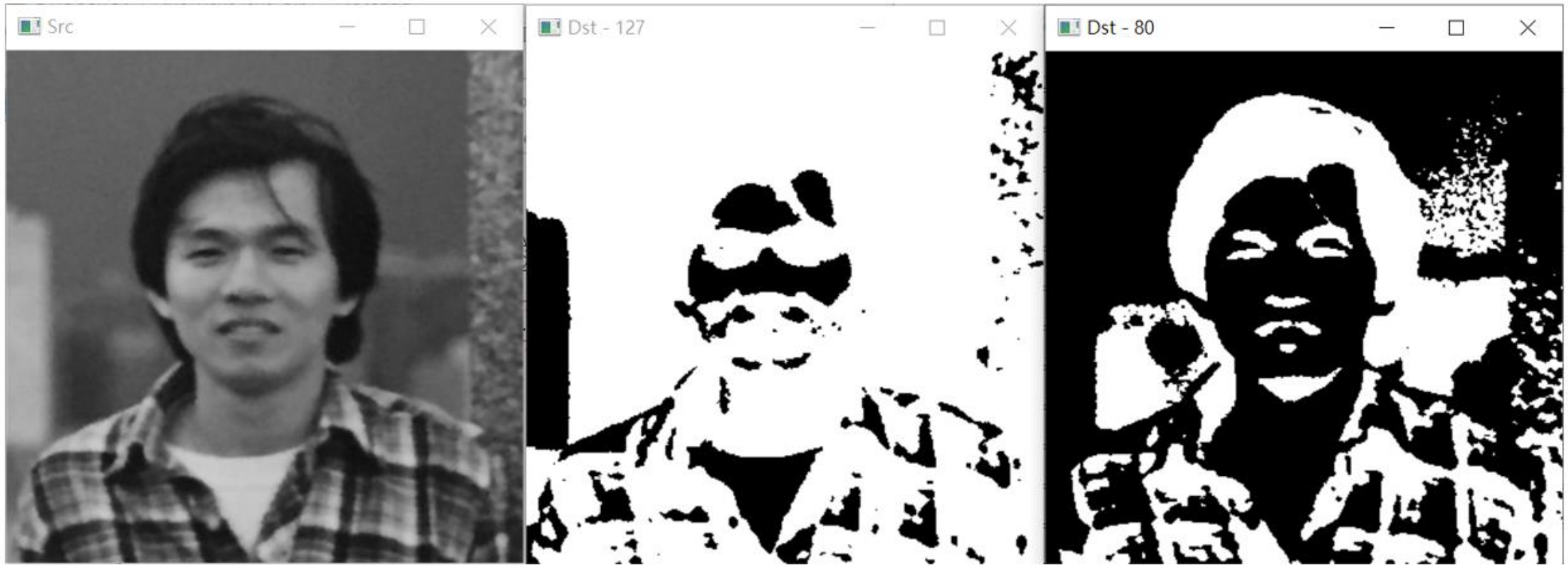
- **9-1-3：反二值化處理THRESH_BINARY_INV**
- if 像素值 > 閾值:
- 　　像素值 = 0 (black)
- 　else:
- 　　像素值 = 最大值
-
- 程式實例：使用THRESH_BINARY_INV，觀察執行結果。

- import cv2
- import numpy as np

- thresh = 127 # 定義閾值
- maxval = 255 # 定義像素最大值
- src = np.random.randint(0,256,size=[3,5],dtype=np.uint8)
- ret, dst = cv2.threshold(src,thresh,maxval,cv2.THRESH_BINARY_INV)
- print(f"src =\n {src}")
- print(f"threshold = {ret}")
- print(f"dst =\n {dst}")

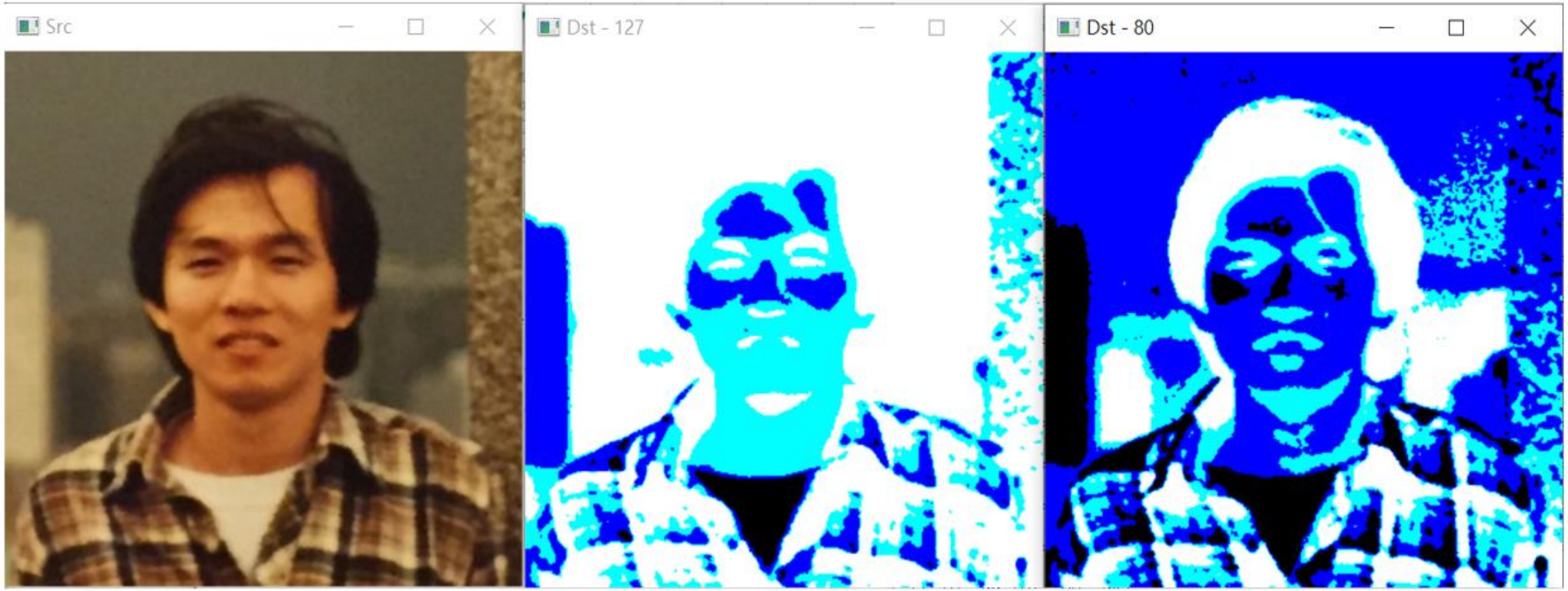
```
src =
 [[ 10  99 161  66  94]
 [203 106 221 162  99]
 [240 136 173 212  26]]
threshold = 127.0
dst =
 [[255 255   0 255 255]
 [  0 255   0   0 255]
 [  0   0   0   0 255]]
```

- 程式實例：使用THRESH_BINARY_INV(反二值化處理)，觀察執行結果。



threshold = 127　　threshold = 80

- 程式實例：使用彩色讀取重新設計。



threshold = 127

threshold = 80

- 程式練習：請讀取左邊原圖，使用二值化處理自行設定閾值，讓結果可以如第二張與第三張之執行結果

9-1-4：截斷閾值處理THRESH_TRUNC

- 如果影像的像素值大於閾值取閾值，其他則像素值不變，語法觀念如下：
- if 像素值 > 閾值:
- 　像素值 = 閾值
- else:
- 　像素值 = 像素值　　# 相當於不變

- 程式實例：使用THRESH_TRUNC，可以觀察執行結果。

```
import cv2
import numpy as np

thresh = 127                    # 定義閾值
maxval = 255                    # 定義像素最大值
src = np.random.randint(0,256,size=[3,5],dtype=np.uint8)
ret, dst =
cv2.threshold(src,thresh,maxval,cv2.THRESH_TRUNC)
print(f"src =\n {src}")
print(f"threshold = {ret}")
print(f"dst =\n {dst}")
```

```
src =
 [[ 86 249 173 107 208]
 [ 37 116  82  55 125]
 [102 156 181  42  64]]
threshold = 127.0
dst =
 [[ 86 127 127 107 127]
 [ 37 116  82  55 125]
 [102 127 127  42  64]]
```

- 程式實例：使用THRESH_TRUNC

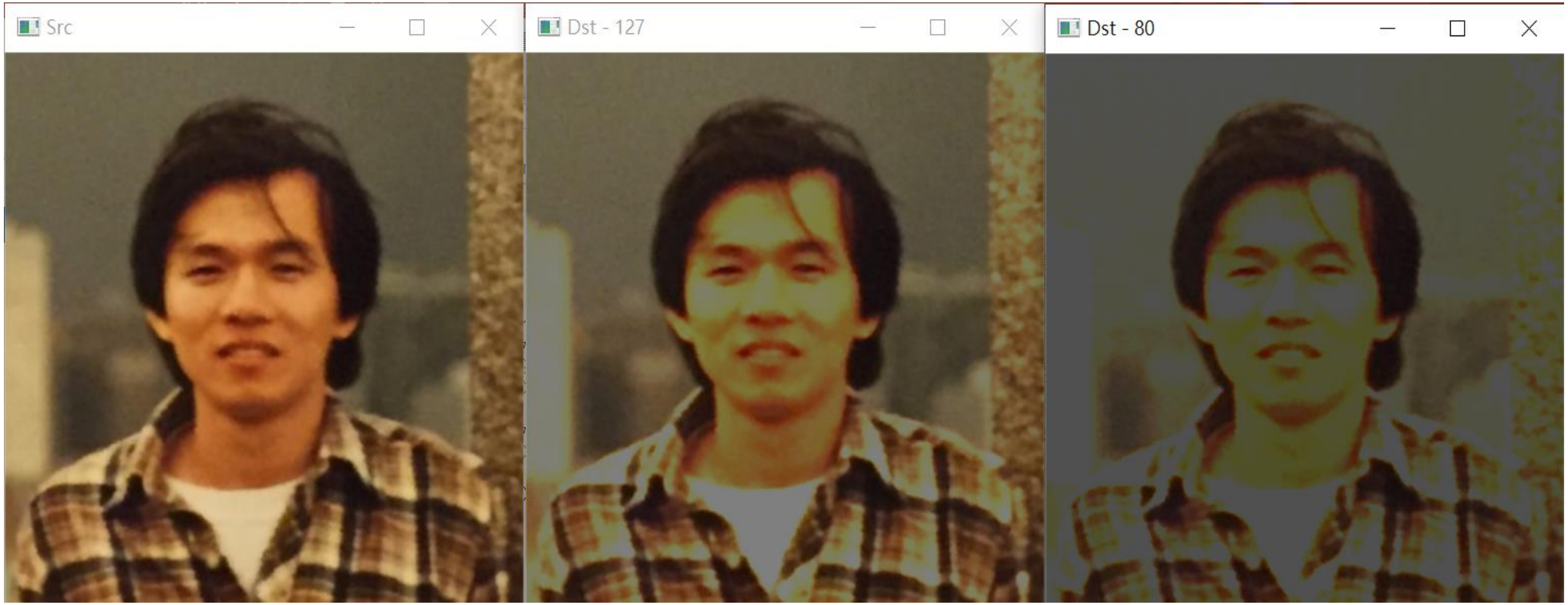
Threshold=127

```
src =
 [[ 86 249 173 107 208]
 [ 37 116  82  55 125]
 [102 156 181  42  64]]
threshold = 127.0
dst =
 [[ 86 127 127 107 127]
 [ 37 116  82  55 125]
 [102 127 127  42  64]]
```

Threshold=80

```
src =
 [[206 175   5 136  31]
 [219 110 178  78 231]
 [232 230 150  27 247]]
threshold = 80.0
dst =
 [[80 80  5 80 31]
 [80 80 80 78 80]
 [80 80 80 27 80]]
```

- 程式實例ch9_11.py：使用彩色讀取重新設計ch9_10.py。



9-1-5：低閾值用0處理THRESH_TOZERO

- 如果影像的像素值**小於或等於**閾值，像素值取0，其他則像素值不變，語法觀念如下：
- if 像素值 **<=** 閾值:
- 像素值 = 0
- else:
- 像素值 = 像素值 # 相當於不變

- 程式實例：使用THRESH_TOZERO，可以觀察執行結果。

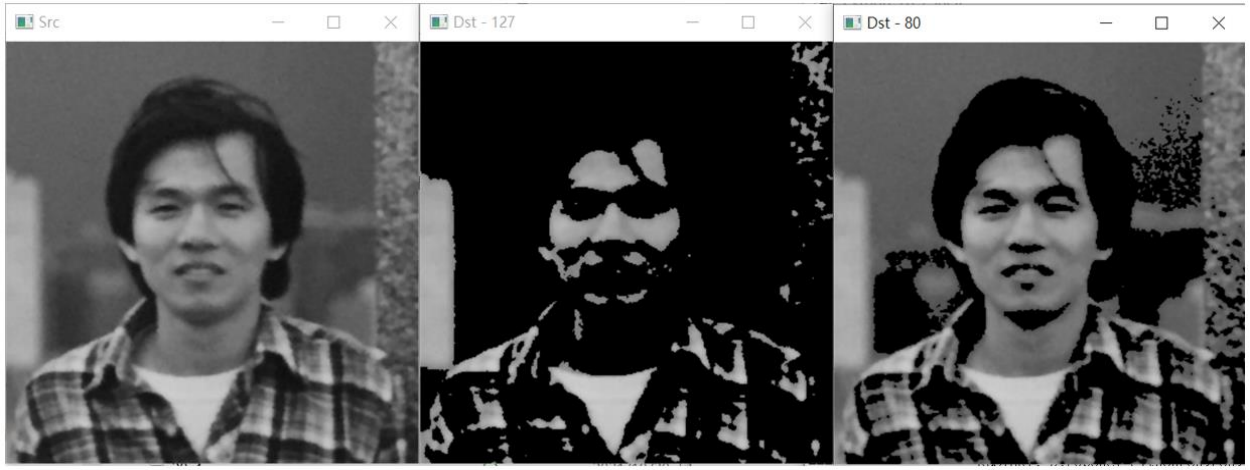
```
import cv2
import numpy as np

thresh = 127                    # 定義閾值
maxval = 255                    # 定義像素最大值
src = np.random.randint(0,256,size=[3,5],dtype=np.uint8)
ret, dst = cv2.threshold(src,thresh,maxval,cv2.THRESH_TOZERO)
print(f"src =\n {src}")
print(f"threshold = {ret}")
print(f"dst =\n {dst}")
```

```
src =
 [[210  33 147   7 223]
 [224  55 227 140 205]
 [ 24 156  98  39 234]]
threshold = 127.0
dst =
 [[210   0 147   0 223]
 [224   0 227 140 205]
 [  0 156   0   0 234]]
```

- 程式實例：使用THRESH_TOZERO，可以觀察執行結果。

```
import cv2
thresh = 127                    # 定義閾值
maxval = 255                    # 定義像素最大值
src = cv2.imread("jk.jpg",cv2.IMREAD_GRAYSCALE)
ret, dst = cv2.threshold(src,thresh,maxval,cv2.THRESH_TOZERO)
cv2.imshow("Src",src)
cv2.imshow("Dst - 127",dst)       # threshold = 127
thresh1 = 80                    # 修訂所定義的閾值
ret1, dst1 = cv2.threshold(src,thresh1,maxval,cv2.THRESH_TOZERO)
cv2.imshow("Dst - 80",dst1)       # threshold = 80

cv2.waitKey(0)
cv2.destroyAllWindows()
```

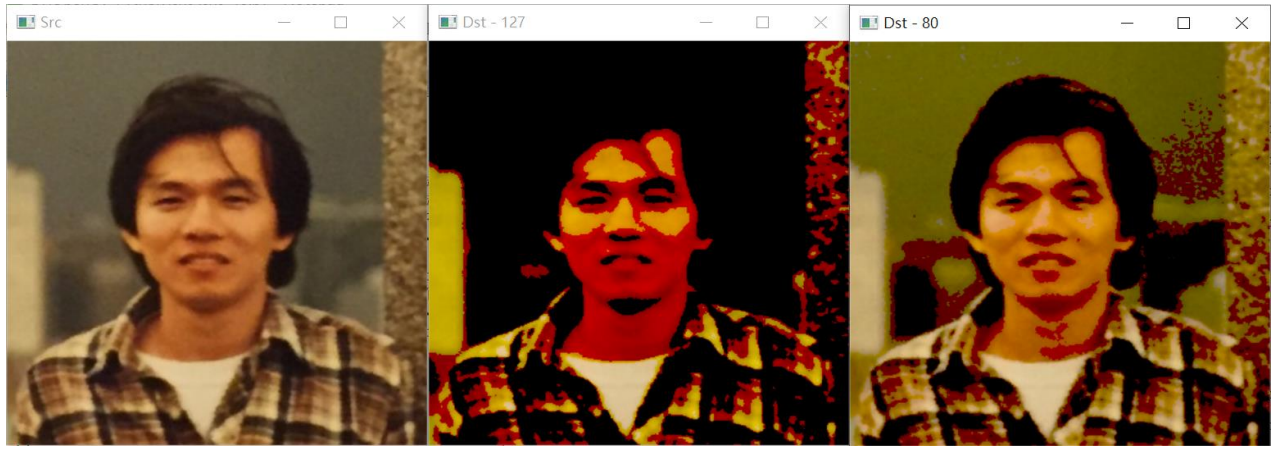
```
if 像素值 <= 閾值:
    像素值 = 0
  else:
    像素值 = 像素值
```



thresh = 127　　thresh = 80

- 程式實例：使用彩色讀取，可以觀察執行結果。

```
import cv2

thresh = 127                    # 定義閾值
maxval = 255                    # 定義像素最大值
src = cv2.imread("jk.jpg")
ret, dst = cv2.threshold(src,thresh,maxval,cv2.THRESH_TOZERO)
cv2.imshow("Src",src)
cv2.imshow("Dst - 127",dst)     # threshold = 127
thresh = 80                     # 修訂所定義的閾值
ret, dst = cv2.threshold(src,thresh,maxval,cv2.THRESH_TOZERO)
cv2.imshow("Dst - 80",dst)      # threshold = 80

cv2.waitKey(0)
cv2.destroyAllWindows()
```



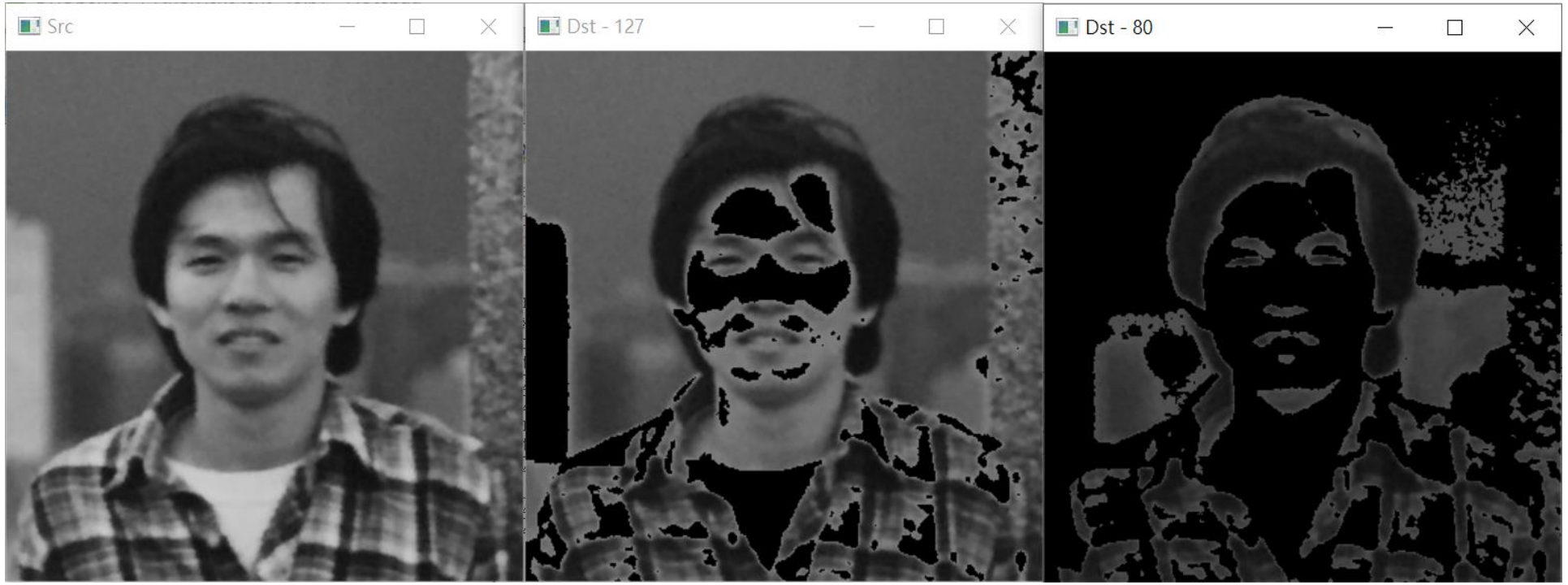
9-1-6：高閾值用0處理THRESH_TOZERO_INV

- 如果影像的像素值大於閾值，像素值取0，其他則像素值不變，語法觀念如下：
- if 像素值 > 閾值:
- 　像素值 = 0
- else:
- 　像素值 = 像素值　　# 相當於不變

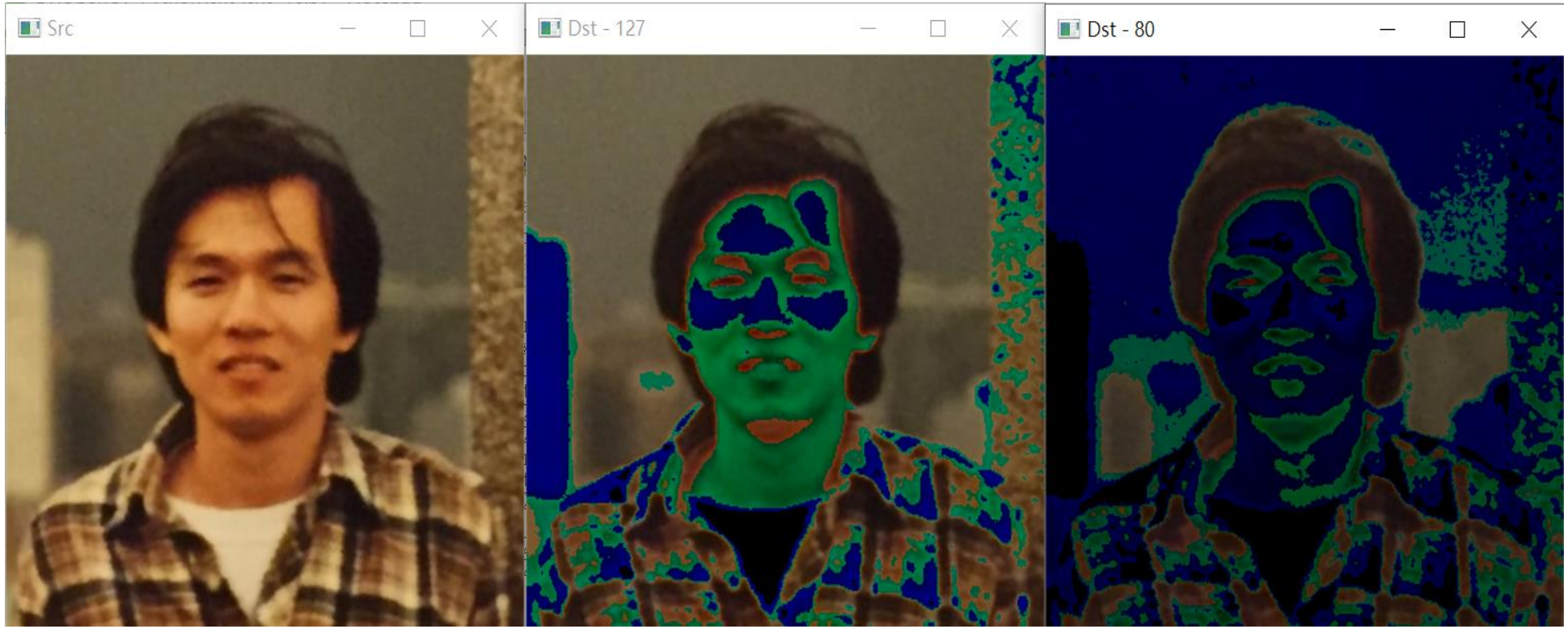
- 程式實例 ch9_15.py：使用 THRESH_TOZERO_INV，重新設計 ch9_12.py，讀者可以觀察執行結果。

```
==================== RESTART: D:/OpenCV_Python/ch9/ch9_15.py ==================
src =
 [[ 51 187  20  74  35]
 [242 157 150 155  21]
 [209  15 241 255 112]]
threshold = 127.0
dst =
 [[ 51   0  20  74  35]
 [  0   0   0   0  21]
 [  0  15   0   0 112]]
```

- 程式實例：使用THRESH_TOZERO_INV，觀察執行結果。



- 程式實例：使用彩色讀取，觀察執行結果。



9-2：Otsu演算法

- 假設有一個影像image內容如下：
- [[108, 108, 120, 120]
- [108, 108, 120, 120]
- [120, 120, 120, 120]]

- 如果我們仍用127當作閾值，這時可以得到下列結果。
- [[0, 0, 0, 0]
- [0, 0, 0, 0]
- [0, 0, 0, 0]]

- 假設有一個影像image內容如下：
- [[108, 108, 120, 120]
- [108, 108, 120, 120]
- [120, 120, 120, 120]]

- 如果我們使用80當作閾值，這時可以得到下列結果。
- [[255, 255, 255, 255]
- [255, 255, 255, 255]
- [255, 255, 255, 255]]

- 很明顯這不是我們所要結果，如果從上述數據判斷，可以將閾值設為108(含)至120(不含)間則是完美的結果，例如：將閾值設為108，可以得到下列比較好的結果。

- [[0, 0, 255, 255]
- [0, 0, 255, 255]
- [255, 255, 255, 255]]

- **Ostu函數，可以根據目前影像，找出最佳的門檻值**

- 將Otsu方法在應用在threshold( )函數。首先，必須將閾值設為0，假設我們使用THRESH_BINARY，則threshold( )函數的語法如下：
- thresh = 0
- maxval = 255
- ret, dst = cv2.threshold(src,thresh,maxval,cv2.THRESH_BINARY+cv2.THRESH_OTSU)
- 上述回傳值ret是Otsu方法計算的閾值，dst則是回傳的影像。

- 程式實例ch9_18.py：，列出原始陣列與測試陣列。
- import cv2
- import numpy as np

- thresh = 127 # 定義閾值
- maxval = 255 # 定義像素最大值
- src = np.ones((3,4),dtype=np.uint8) * 120 # 設定陣列是 120
- src[0:2,0:2]=108 # 設定陣列區間為 0
- ret, dst = cv2.threshold(src,thresh,maxval,cv2.THRESH_BINARY)
- print(f"src =\n {src}")
- print(f"threshold = {ret}")
- print(f"dst =\n {dst}")

```
====================
src =
 [[108 108 120 120]
 [108 108 120 120]
 [120 120 120 120]]
threshold = 127.0
dst =
 [[0 0 0 0]
 [0 0 0 0]
 [0 0 0 0]]
```

- 程式實例：測試Otsu方法所回傳的陣列與閾值。

```
import cv2
import numpy as np

thresh = 0                          # 定義閾值
maxval = 255                        # 定義像素最大值
src = np.ones((3,4),dtype=np.uint8) * 120   # 設定陣列是 120
src[0:2,0:2]=108                    # 設定陣列區間為 0
ret, dst = cv2.threshold(src,thresh,maxval,cv2.THRESH_BINARY+cv2.THRESH_OTSU)
print(f"src =\n {src}")
print(f"threshold = {ret}")
print(f"dst =\n {dst}")
```

```
src =
 [[108 108 120 120]
 [108 108 120 120]
 [120 120 120 120]]
threshold = 108.0
dst =
 [[  0   0 255 255]
 [  0   0 255 255]
 [255 255 255 255]]
```

得到Otsu方法的閾值是108.0，這也是完美分割的結果。

- 程式實例ch9_20.py：使用閾值是127，和使用Otsu方法，程式也同時列出Otsu方法的閾值。

```
==================== RESTART: D:/OpenCV_Python/ch9/ch9_20.py ==================
threshold = 107.0
```

thresh = 127　　Otsu方法

9-3：自適應閾值方法adaptiveThreshhold( )函數

- 一般的二值化只會考慮單一點的值，直接去做閥值分析，但一張圖片的「每個鄰近的像素都是彼此有關連的」。如果單純只針對單一個點去看，失去了對整張圖相鄰點的考慮。
- 因此自適應二值化可以計算每個像素點與周圍區域的關係，然後再利用此閾值對周遭的區域進行處理，此方法可以獲得更好的影像效果。

- 計算閾值與周圍區域的關係有兩種演算法：

❑ ADAPTIVE_THRESH_MEAN_C：算術平均法，周圍區域的平均值當作閾值，再減去參數C，C是一個常數下列語法會解說。

❑ ADAPTIVE_THRESH_GAUSSIAN_C：高斯加權和法，這是一種高斯加權的平均法，計算中心點與周圍區域所有像素點進行加權計算，計算結果再減去參數C，C是一個常數下列語法會解說。

- dst = cv2.adaptiveThreshold(src, maxValue, adaptiveMethod, thresholdType, blockSize, C)

- maxValue :最大像素值
- adaptiveMethod :
  - ADAPTIVE_THRESH_MEAN_C 算術平均法
  - ADAPTIVE_THRESH_GAUSSIAN_C 高斯加權平均法
- thresholdType: 閾值類型，THRESH_BINARY OR THRESH_BINARY_INV
- blockSize : 用來計算閾值的周圍區域大小, 一般3, 5, 7 ....單數
- C :偏移量,微調用的參數, 通常是正數, 也可以是零或負數

- 程式實例：對一座美國大學的建築物，使用二值化處理threshold( )和adaptiveThreshold( )自適應閾值法，在使用自適應閾值方法時，需同時使用ADAPTIVE_THRESH_MEAN_C方法和ADAPTIVE_THRESH_GAUSSIAN_C方法然後列出處理結果。

原始影像　　二值化處理THRESH_BINARY

ADAPTIVE_THRESH_MEAN_C　　ADAPTIVE_THRESH_GAUSSIAN_C

- import cv2

- src = cv2.imread("school.jpg",cv2.IMREAD_GRAYSCALE) # 灰階讀取
- thresh = 127 # 閾值
- maxval = 255 # 定義像素最大值
- ret,dst = cv2.threshold(src,thresh,maxval,cv2.THRESH_BINARY) # 二值化處理
- # 自適應閾值計算方法為ADAPTIVE_THRESH_MEAN_C
- dst_mean = cv2.**adaptiveThreshold**(src,maxval,cv2.ADAPTIVE_THRESH_MEAN_C, cv2.THRESH_BINARY,3,5)
- # 自適應閾值計算方法為ADAPTIVE_THRESH_GAUSSIAN_C
- dst_gauss = cv2.adaptiveThreshold(src,maxval,cv2.ADAPTIVE_THRESH_GAUSSIAN_C,cv2.THRESH_BINARY,3,5)
- cv2.imshow("src",src) # 顯示原始影像
- cv2.imshow("THRESH_BINARY",dst) # 顯示二值化處理影像
- cv2.imshow("ADAPTIVE_THRESH_MEAN_C",dst_mean) # 顯示自適應閾值結果
- cv2.imshow("ADAPTIVE_THRESH_GAUSSIAN_C",dst_gauss) # 顯示自適應閾值結果

- cv2.waitKey(0)
- cv2.destroyAllWindows()

9-4：平面圖的分解

影像像素值二進化的分解：

$$img = a_7 * 2^7 + a_6 * 2^6 + a_5 * 2^5 + a_4 * 2^4 + a_3 * 2^3 + a_2 * 2^2 + a_1 * 2^1 + a_0 * 2^0$$

201	88	90
123	12	36
79	6	200

		a_7 a_4 a_1 a_0
11001001	01011000	01011010
01111011	00001100	00100100
01001111	00000110	11001000

1	0	0
1	0	0
1	0	0

a_0

0	0	1
1	0	0
1	1	0

a_1

0	0	0
0	1	1
1	1	0

a_2

1	1	1
1	1	0
1	0	1

a_3

0	1	1
1	0	0
0	0	0

a_4

0	0	0
1	0	1
0	0	0

a_5

1	1	1
1	0	0
1	0	1

a_6

1	0	0
0	0	0
0	0	1

a_7

0	1	1
1	0	0
0	0	0

a_4

0	0	0
1	0	1
0	0	0

a_5

1	1	1
1	0	0
1	0	1

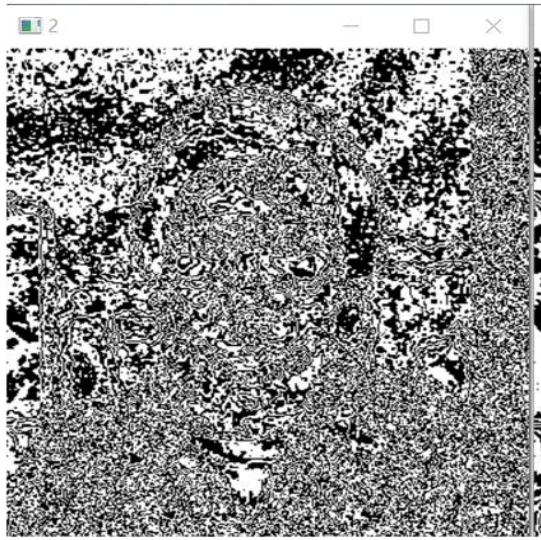
a_6

1	0	0
0	0	0
0	0	1

a_7

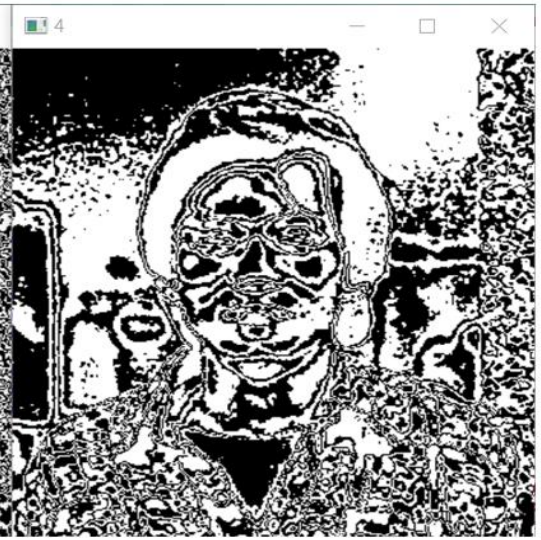
程式實例 ch9_22.py：將原始圖案分解，依照位元觀念，從 a_0 至 a_7 分解為 8 個圖案，在分解過程，將非 0 的像素值改為 255，也就是本章的閾值觀念，最後顯示 8 個影像圖。

原始影像　　a_0　　a_1

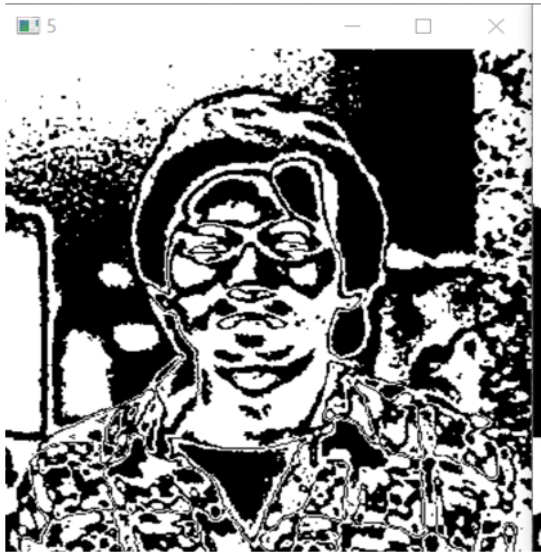


a_2

a_3



a_4



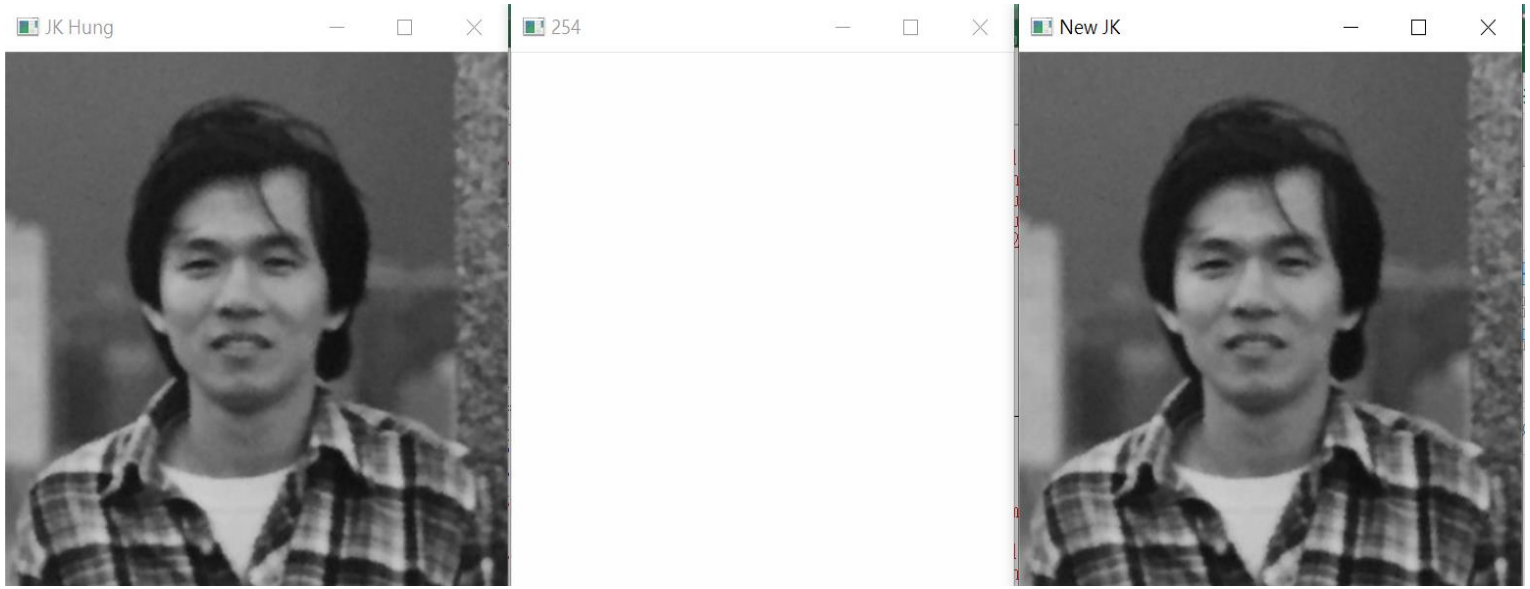
a_5

a_6

a_7

9-5：隱藏在影像內的數位浮水印

- **9-5-1：驗證最低有效位元對影像沒有太大的影響**
- 程式實例：驗證最低有效位元對影像沒有太大的影響，將jk.jpg的最低有效位元設為0，同時觀察執行結果。



```
import cv2
import numpy as np

jk = cv2.imread("jk.jpg",cv2.IMREAD_GRAYSCALE)
cv2.imshow("JK Hung",jk)                          # 顯示原始影像

row, column = jk.shape                            # 取得列高和欄寬
h7 = np.ones((row,column),dtype=np.uint8) * 254   # 建立像素值是254的影像
cv2.imshow("254",h7)                              # 顯示像素值是254的影像
new_jk = cv2.bitwise_and(jk,h7)                   # 原始影像最低有效位元是 0
cv2.imshow("New JK",new_jk)                       # 顯示新影像

cv2.waitKey(0)
cv2.destroyAllWindows()
```

9-5-2：建立數位浮水印

- 上述可知:a_0為影像由每個像素點最低有效位元(Least significiant bit, LSB)所組成
- 一般會將數位浮水印隱藏在a_0平面影像, 這樣既不會影響原始影像, 同時也達到隱藏影像的目的；數位浮水印可應用於隱藏版權， 身份驗證或是數位戰爭隱藏情報資訊

- 建立數位浮水印包含6個步驟：
  - 1：取得原始影像的row和column。
  - 2：建立像素值是254的提取矩陣。
  - 3：取得原始影像的高7位影像。
  - 4：建立與原始影像相同大小的浮水印影像。
  - 5：將浮水印影像嵌入原始影像，建立含浮水印的影像，稱結果影像。
  - 6：從結果影像擷取浮水印影像。

取得原始影像的row和column

- jk = cv2.imread(“jk.jpg”,cv2.IMREAD_GRAYSCALE)
- row, column = jk.shape # 取得列高與行寬

建立像素值是254的提取矩陣

- h7 = np.ones((row, column), dtype=np.uint8) * 254

建立浮水印影像

- 要建立浮水印影像需要2個步驟：
- 1：建立浮水印大小是row x column的影像矩陣。
- 2：浮水印影像是灰階影像時，需做二值化處理。

255	255	0	0
0	255	255	0
255	255	0	255

1	1	0	0
0	1	1	0
1	1	0	1

00000001	00000001	00000000	00000000
00000000	00000001	00000001	00000000
00000001	00000001	00000000	00000001

- 假設浮水印影像是copyright.jpg，下列是處理浮水印的語法。
- watermark = cv2.imread(“copyright.jpg”,IMREAD_GRAYSCALE)
- ret, wm = cv2.threshold(watermark,0,1,cv2.THRESH_BINARY)

255	255	0	0
0	255	255	0
255	255	0	255

1	1	0	0
0	1	1	0
1	1	0	1

00000001	00000001	00000000	00000000
00000000	00000001	00000001	00000000
00000001	00000001	00000000	00000001

9-5-7：將浮水印影像嵌入原始影像

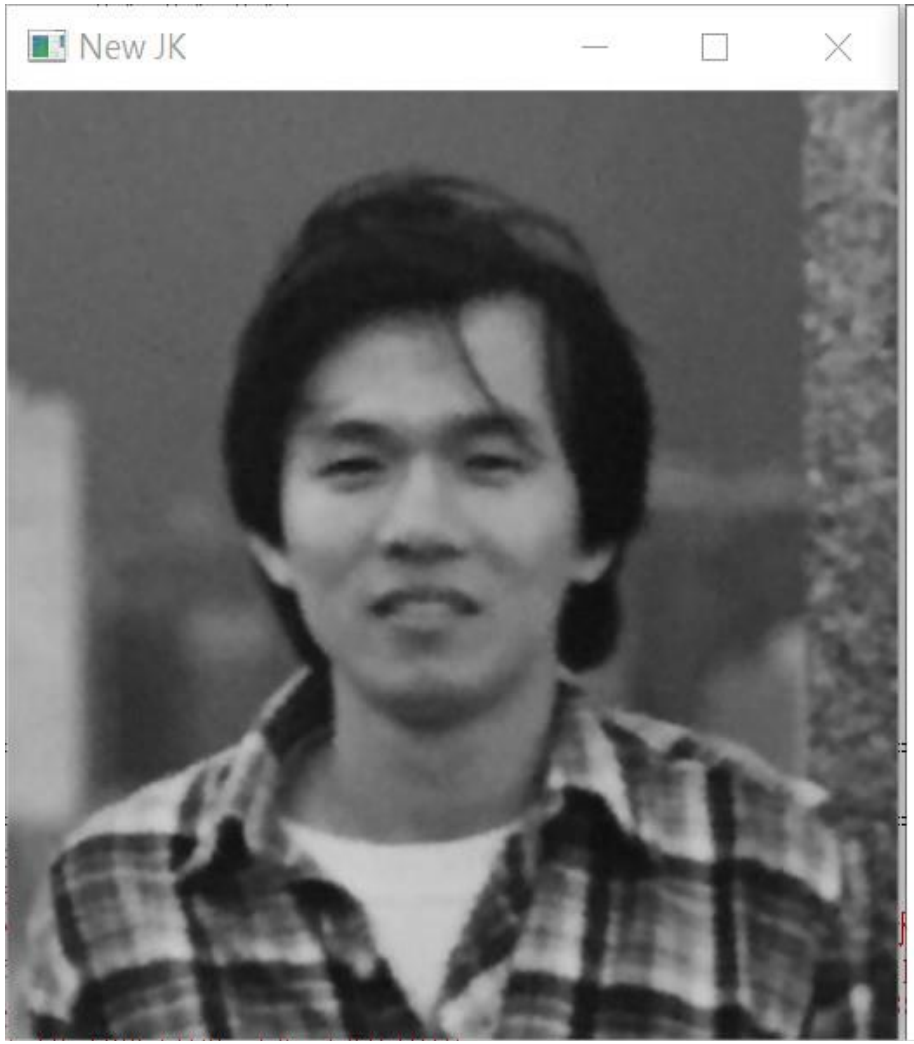
- new_jk = cv2.bitwise_or(tmp_jk, wm)
- 程式實例：將浮水印影像嵌入原始影像。

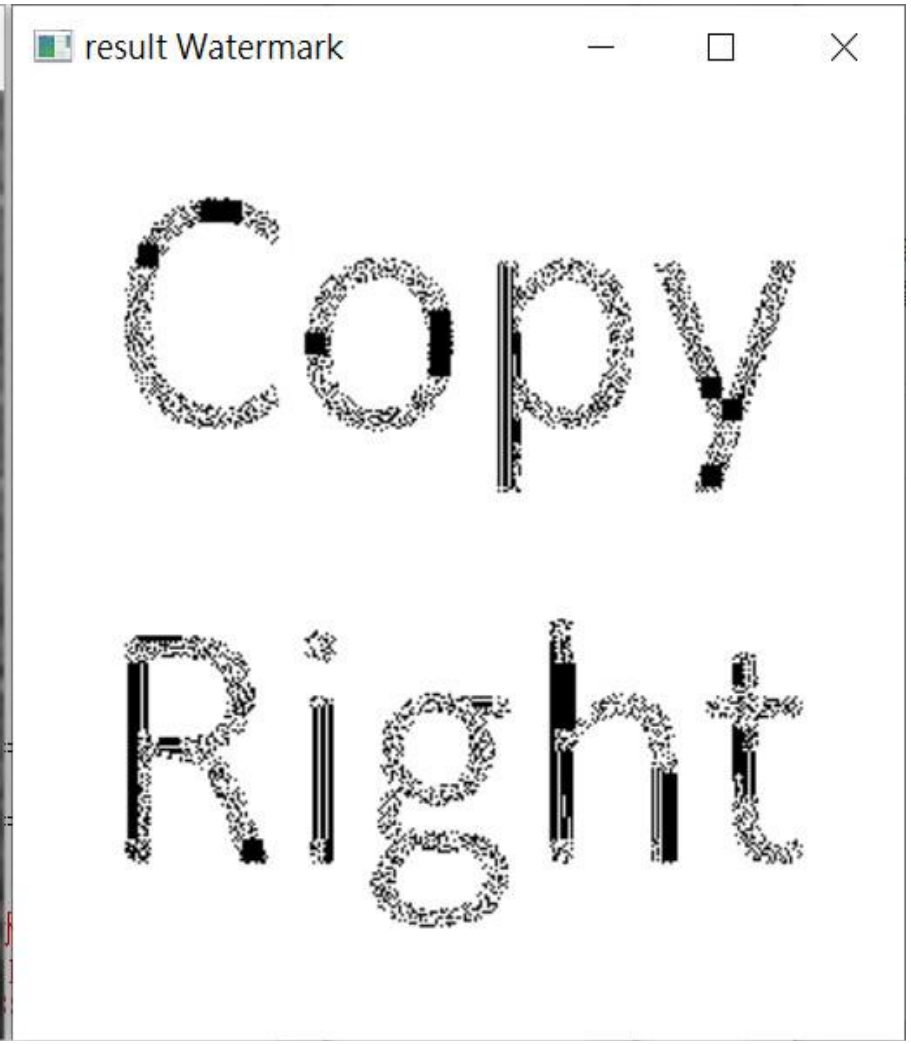
```
import cv2
import numpy as np

jk = cv2.imread("jk.jpg",cv2.IMREAD_GRAYSCALE)
cv2.imshow("JK Hung",jk)                    # 顯示原始影像

row, column = jk.shape                      # 取得列高和欄寬
h7 = np.ones((row,column),dtype=np.uint8) * 254    # 建立像素值是254的影像
tmp_jk = cv2.bitwise_and(jk,h7)             # 原始影像最低有效位元是 0
watermark = cv2.imread("copyright.jpg",cv2.IMREAD_GRAYSCALE)
cv2.imshow("Copy Right",watermark)          # 顯示浮水印影像
ret, wm = cv2.threshold(watermark,0,1,cv2.THRESH_BINARY)
# 浮水印影像嵌入最低有效位元是 0的原始影像
new_jk = cv2.bitwise_or(tmp_jk, wm)
cv2.imshow("New JK",new_jk)                 # 顯示新影像

cv2.waitKey(0)
cv2.destroyAllWindows()
```

9-5-8：擷取浮水印影像

- h0 = np.ones((row, column), dtype=np.uint8)
- 使用指令將從含浮水印的new_jk擷取浮水印。
- wm = cv2.bitwise_and(new_jk, h0)
- 接著將大於閾值(0)的值設為255，下列dst就是我們想要的浮水印內容。
- ret, dst = cv2.threshold(wm,0,255,cv2.THRESH_BINARY)

- 程式實例：將浮水印影像copyright.jpg，嵌入原始影像，然後再將此影像擷取出來。


New JK


result Watermark
Copy
Right